

From Notebook to MLOps Pipeline

A Practical Technical Case Study with Apache Airflow and Docker

Building an end-to-end machine learning workflow from data validation to automated model training

Python	Apache Airflow	Docker
scikit-learn	Random Forest	MLOps

Technical Case Study & Beginner-Friendly Implementation Guide

Portfolio project: Screen Time Prediction

Contents

1. Why this guide exists
2. What we build
3. Project architecture
4. From notebook to reusable ML code
5. Data validation and feature engineering
6. Train/test strategy and modeling
7. Model artifacts
8. Apache Airflow orchestration
9. Docker execution
10. Successful end-to-end run
11. Problems solved and lessons learned
12. What makes this MLOps?
13. GitHub structure
14. Interview explanation
15. Production readiness and next steps
16. Key takeaway

Audience: This guide is designed for readers who understand basic Python or machine learning but are new to MLOps. It explains both the implementation and the reason behind each step.

1. Why this guide exists

A machine learning notebook can prove that a model works, but it does not yet show how that model can be run repeatedly, checked, automated, and maintained as part of a workflow.

This case study follows a small screen-time prediction project from an interactive Jupyter Notebook to an automated MLOps training pipeline. The focus is deliberately practical: what each component does, why it exists, and how the components connect.

Core transition: Notebook experiment -> reusable Python training code -> saved model artifacts -> Airflow orchestration -> Dockerized execution.

2. What we build

The project predicts mobile application usage in minutes from screen-time behavior. The final workflow separates data checks, feature preparation, model training, and metric reporting into independent steps.

Column	Meaning
Date	Observation date
App	Mobile application
Usage (minutes)	Target variable
Notifications	Number of notifications
Times Opened	Number of app openings

validate_raw_data -> prepare_features -> train_model -> report_metrics

3. Project architecture

From Notebook to an Automated MLOps Training Pipeline

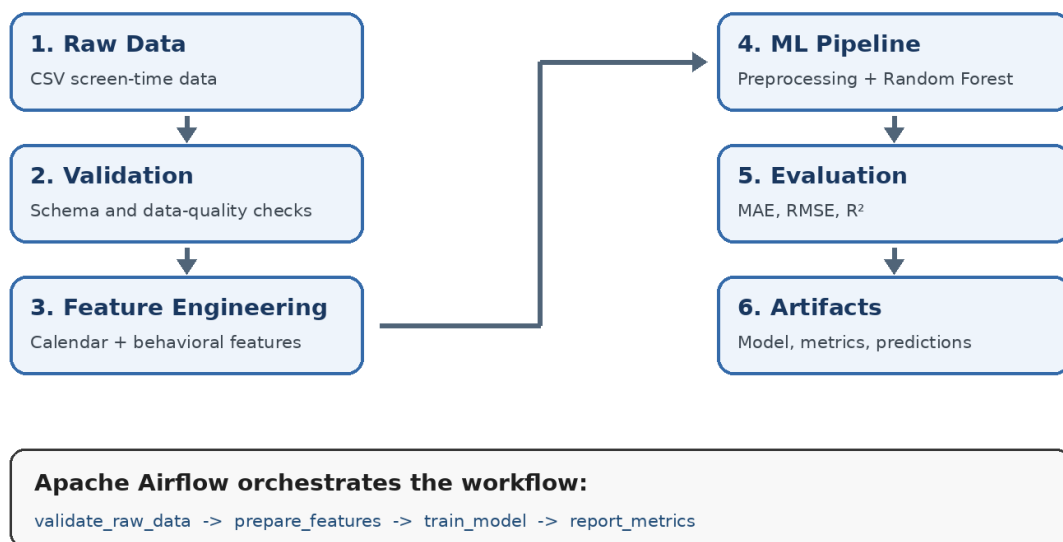


Figure 1. From raw data to an orchestrated MLOps training workflow.

- Notebook layer: exploration, visualization, experimentation, and explanation.
- Reusable ML layer: functions in train_pipeline.py that can execute without Jupyter.
- Orchestration layer: an Airflow DAG that controls task order, scheduling, status, and logs.
- Runtime layer: Docker Compose provides a consistent local Airflow environment.

4. From notebook to reusable ML code

The notebook is the right place to understand the data, create charts, test ideas, and compare model behavior. It is not the ideal place for code that must be called automatically by another system.

The project therefore keeps exploration in the notebook and moves repeatable logic into `train_pipeline.py`.

```
load_data()
validate_data()
engineer_features()
split_by_date()
build_model()
evaluate_model()
train_and_save()
```

Why this separation matters: A scheduler such as Airflow should call stable Python functions, not depend on a human opening a notebook and pressing Run All.

5. Data validation and feature engineering

Before training, the pipeline checks whether the input is structurally usable. This creates a simple quality gate.

- Required columns are present.
- Missing values are counted.
- Exact duplicate rows are counted.
- Negative values in numeric fields are rejected.
- Repeated Date + App combinations are monitored.

Feature engineering is intentionally transparent and reproducible:

```
data["DayOfWeek"] = data["Date"].dt.dayofweek
data["DayOfMonth"] = data["Date"].dt.day
data["IsWeekend"] = (data["DayOfWeek"] >= 5).astype(int)
data["Notifications_x_TimesOpened"] = data["Notifications"] * data["Times Opened"]
```

Methodological choice: The dataset contains repeated Date + App combinations. A simple row-based `shift(1)` would not reliably mean "previous calendar day" for the same app, so the project avoids that ambiguous lag feature.

6. Train/test strategy and modeling

Instead of a normal random row split, the project groups observations by Date with `GroupShuffleSplit`. This prevents the same date from appearing in both training and test data.

Result: The test set represents unseen days, reducing same-day leakage between train and test.

Preprocessing and modeling are combined in one scikit-learn Pipeline:

- Numeric features -> MinMaxScaler
- Categorical App feature -> OneHotEncoder(handle_unknown="ignore")
- Estimator -> RandomForestRegressor

```
Pipeline(steps=[  
    ("preprocessor", preprocessor),  
    ("model", RandomForestRegressor(...)),  
])
```

Metric	Result	Interpretation
MAE	~16.84 min	Average absolute prediction error
RMSE	~21.48 min	Penalizes larger errors more strongly
R ²	~0.373	Share of target variance explained

How to read the result: The dataset has only 200 rows. The main objective is therefore not to present a state-of-the-art accuracy benchmark, but to demonstrate a sound, repeatable training architecture.

7. Model artifacts

A model that exists only in notebook memory disappears when the kernel stops. The training pipeline therefore saves outputs that can be inspected or reused later.

```
artifacts/  
|-- screentime_random_forest.joblib  
|-- metrics.json  
`-- test_predictions.csv
```

Artifact	Purpose
screentime_random_forest.joblib	Saved preprocessing + trained Random Forest pipeline
metrics.json	Model metadata, validation summary, and evaluation metrics
test_predictions.csv	Actual and predicted values for the test set

This is a lightweight introduction to artifact management. Larger MLOps platforms typically extend this idea with model registries, versioning, lineage, approval states, and rollback mechanisms.

8. Apache Airflow orchestration

Airflow is not the model. Airflow is the orchestrator: it decides what runs, in which order, when it runs, whether it succeeded, and where its logs can be inspected.

```
@dag(...)
def screentime_mlops_pipeline():
    raw_path = validate_raw_data()
    processed_path = prepare_features(raw_path)
    metadata = train_model(raw_path, processed_path)
    report_metrics(metadata)
```

Task	Responsibility
validate_raw_data	Load raw CSV and run data-quality checks
prepare_features	Create reproducible engineered features
train_model	Call reusable training code and save artifacts
report_metrics	Write MAE, RMSE and R ² to task logs

Why task separation matters: When a pipeline fails, Airflow shows exactly which step failed. Downstream tasks can be blocked automatically, and each task has its own status and log.

9. Docker execution

Airflow uses several services. Docker Compose makes it practical to run those services locally and consistently, especially on Windows where native Airflow execution is not the preferred setup.

```
docker compose up airflow-init
docker compose up -d
docker compose ps
```

The Airflow web interface is then opened at:

```
http://localhost:8080
```

Local-development note: This project uses additional Python packages in the Airflow containers for a learning/demo environment. A production implementation should normally build a custom Airflow image with pinned dependencies.

10. Successful end-to-end run

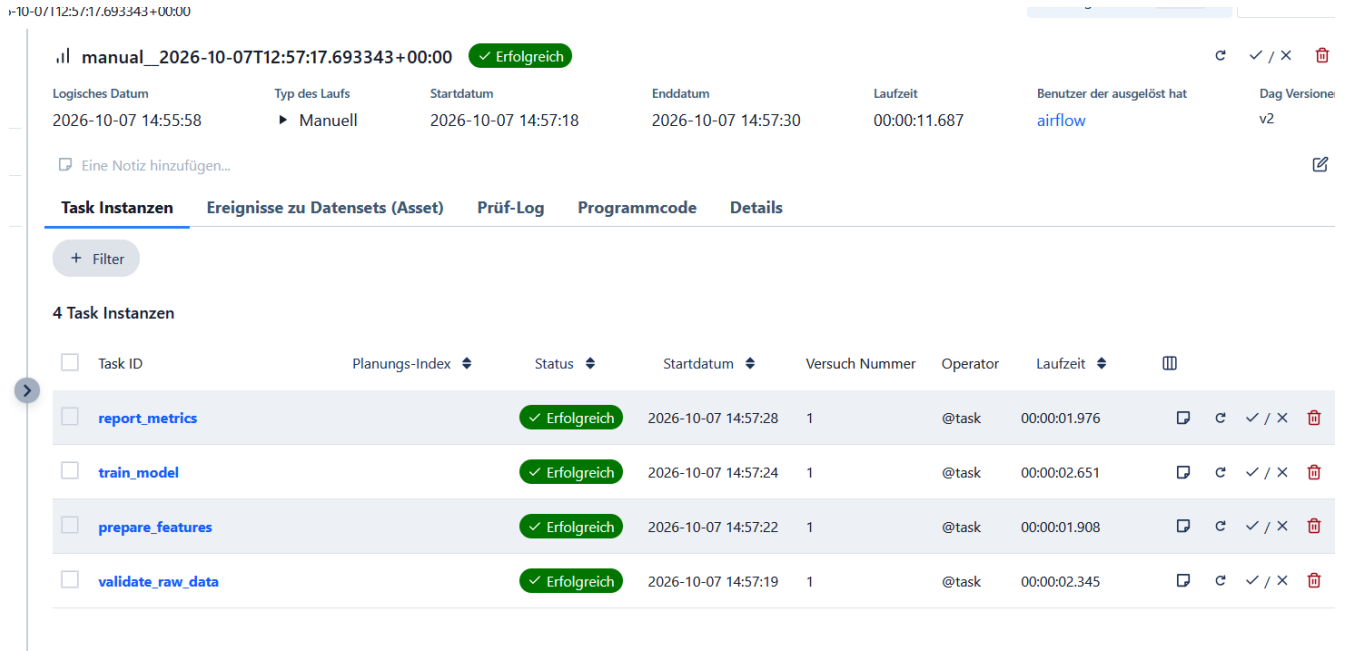


Figure 2. Final Airflow run: all four tasks completed successfully.

The final manual DAG run completed the full workflow successfully in roughly 12 seconds in the local environment. This confirms that the training process can run independently of the notebook.

11. Problems solved and lessons learned

Jupyter checkpoint files were detected as DAGs

Jupyter automatically created .ipynb_checkpoints. Airflow scanned a checkpoint copy of the DAG and calculated the wrong project root, so it searched for the CSV in the checkpoint directory.

Recursive DAG-directory loop

Because the whole project folder was mounted into the container, Airflow also scanned generated runtime directories such as logs. This caused a recursive directory-walking error.

The permanent local fix was an .airflowignore file:

```
.ipynb_checkpoints/  
logs/  
config/  
plugins/  
artifacts/  
__pycache__/
```

Engineering lesson: MLOps is not only model selection. File layout, mounts, runtime isolation, dependencies, logs, and orchestration configuration are part of the solution.

12. What makes this MLOps?

Machine-learning activity	MLOps improvement
Load data manually	Repeatable loading function
Inspect data once	Automated validation gate
Transform data in cells	Reusable feature-engineering function
Train interactively	Reusable train_and_save() function
View metrics only in notebook	Persist metrics as JSON
Keep model in memory	Persist complete trained pipeline
Run steps manually	Airflow orchestration
Depend on local setup	Dockerized Airflow environment
Debug one script	Task-level status and logs

The central learning outcome is the transition from a one-time modeling exercise to a repeatable, observable workflow.

13. Recommended GitHub structure

```
MLOps_Screentime_Prediction/
|-- MLOps_Screentime_Prediction.ipynb
|-- train_pipeline.py
|-- screentime_mlops_dag.py
|-- screentime_analysis.csv
|-- docker-compose.yaml
|-- .airflowignore
|-- README.md
|-- artifacts/
|   |-- metrics.json
|   |-- screentime_random_forest.joblib
|   `-- test_predictions.csv
`-- docs/
    `-- From_Notebook_to_MLOps_Pipeline_Guide.pdf
```

Generated runtime folders such as logs, config, plugins, __pycache__, and .ipynb_checkpoints should normally be excluded from Git.

14. How to explain the project in an interview

60-90 second version: I built an end-to-end MLOps training pipeline for screen-time prediction. I started with a Jupyter Notebook for data exploration and modeling, then moved the reusable logic into a Python training module. I added data validation, reproducible feature engineering, a scikit-learn preprocessing and Random Forest pipeline, model evaluation, and artifact persistence. Finally, I orchestrated the workflow with Apache Airflow and ran it in Docker. The DAG separates validation, feature preparation, training, and metric reporting into independent tasks, and I verified the complete workflow successfully in the Airflow UI.

For Senior, Lead, or MLOps Engineer roles, the strongest message is not the Random Forest algorithm itself. The stronger message is the engineering discipline around repeatability, separation of concerns, orchestration, observability, and environment management.

15. Production readiness and next steps

This project should be described as a production-oriented MLOps prototype, not as a complete production platform.

- MLflow for experiment tracking and model registry.
- A custom Docker image with pinned dependencies.
- FastAPI for an online prediction endpoint.
- GitHub Actions for CI/CD.
- Unit and integration tests.
- Data-drift and model-drift monitoring.
- Production object storage, database, and secrets management.
- Model approval, version promotion, rollback, and governance.

16. Key takeaway

MLOps starts when a model stops being a one-time notebook result and becomes a repeatable, testable, observable, and automated workflow.

This project demonstrates that transition from beginning to end with a small dataset and a deliberately understandable architecture.

References and further reading

- Aman Kharwal, “MLOps Pipeline using Apache Airflow” (project inspiration): <https://amanxai.com/2025/01/20/mlops-pipeline-using-apache-airflow/>
- Apache Airflow documentation: <https://airflow.apache.org/docs/>
- scikit-learn Pipeline and composition: <https://scikit-learn.org/stable/modules/compose.html>
- Docker Compose documentation: <https://docs.docker.com/compose/>